# DL and RL

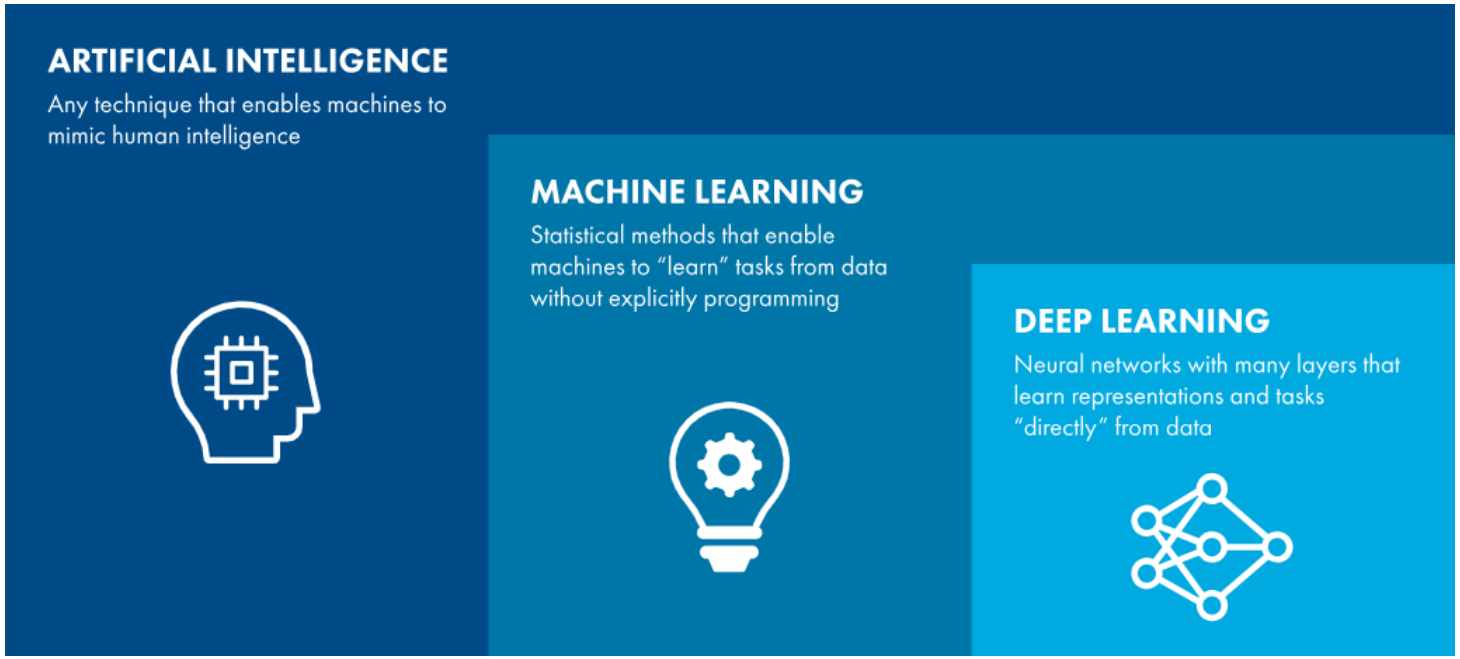
# Index

1. Deep Learning

2. Basic Network Structure

3. Reinforcement Learning

## 1.1. Define of Deep Learning




- DL은 ML의 한 분야로서, 인공 신경망을 깊은 층으로 구성해 데이터를 학습함
- DL은 특정한 목적(Classification, regression, etc..)에 맞게 인공 신경망을 구성하여 다수의 데이터를 통해 학습함으로써, 해당 목적을 달성할 수 있도록 함
- 즉 DL은 구축된 신경망을 데이터를 사용하여 학습하는 기술로, 기존 사람이 규칙을 만드는 방식에서 컴퓨터가 데이터에서 규칙을 스스로 찾아내는 방법임

## ➢ 1.2. Applications in Deep Learning (Computer Vision)

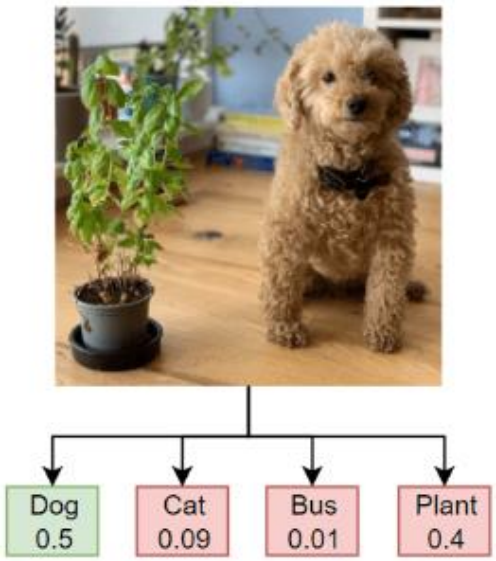
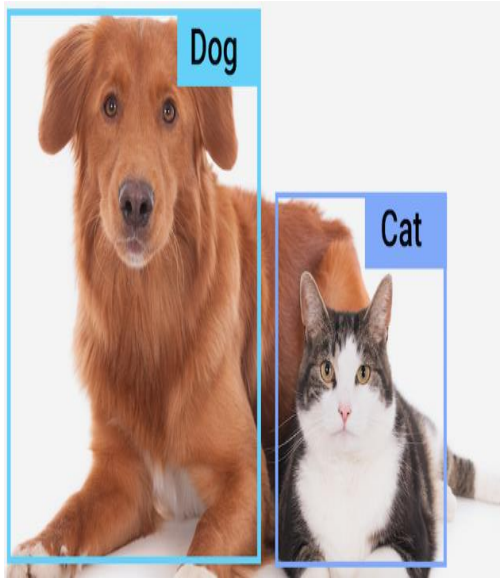
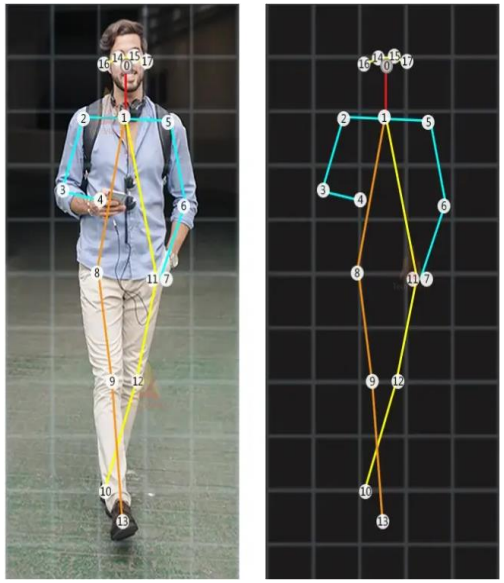



**Image Classification**



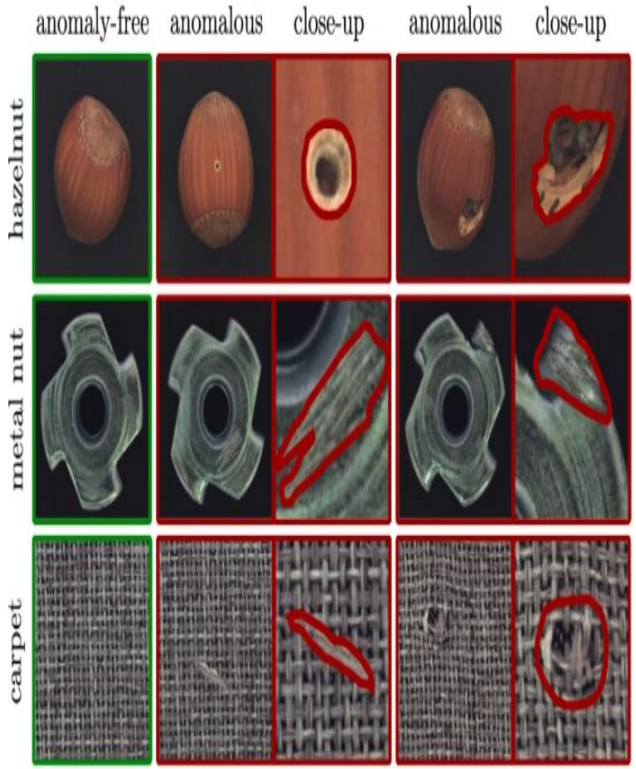

**Object Detection**



**Pose Estimation**

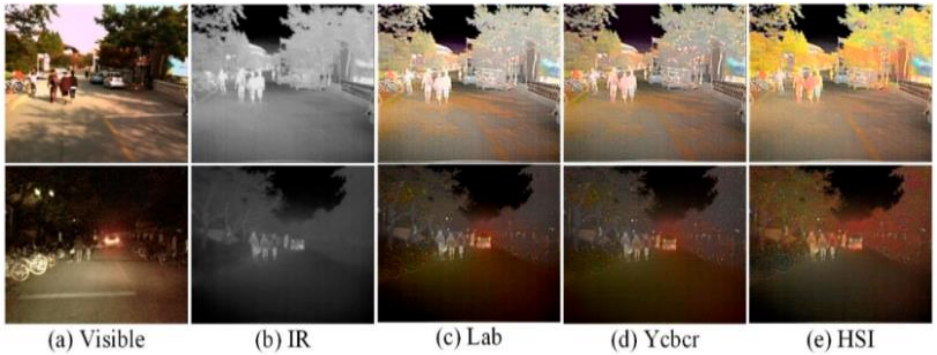
- Image Classification: 이미지 상에 존재하는 객체가 어떤 것인가를 판단
- Object Detection: 이미지 상에 존재하는 객체가 어떤 것이며 어디에 위치하고 있는지를 판단
- Pose Estimation: 이미지 상의 객체의 key point를 추정

## 1.2. Applications in Deep Learning (Computer Vision)




**Anomaly Detection**

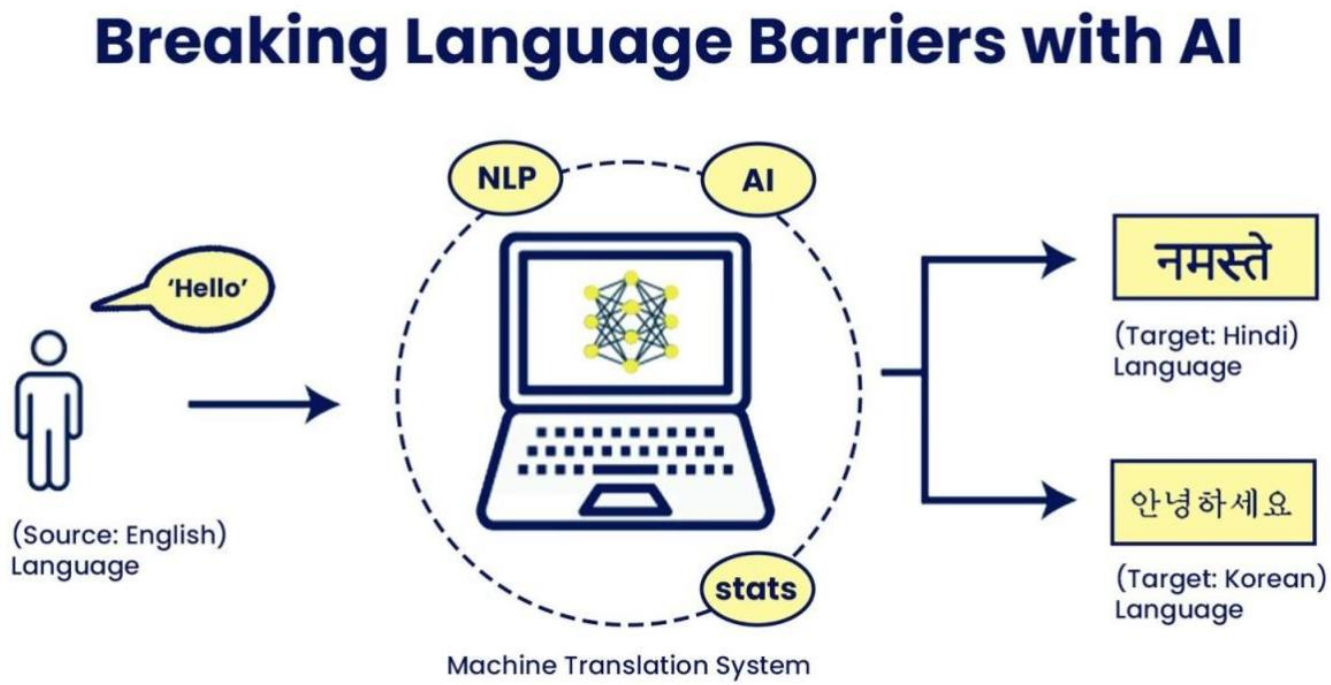
**Image Segmentation**




**Sensor fusion**

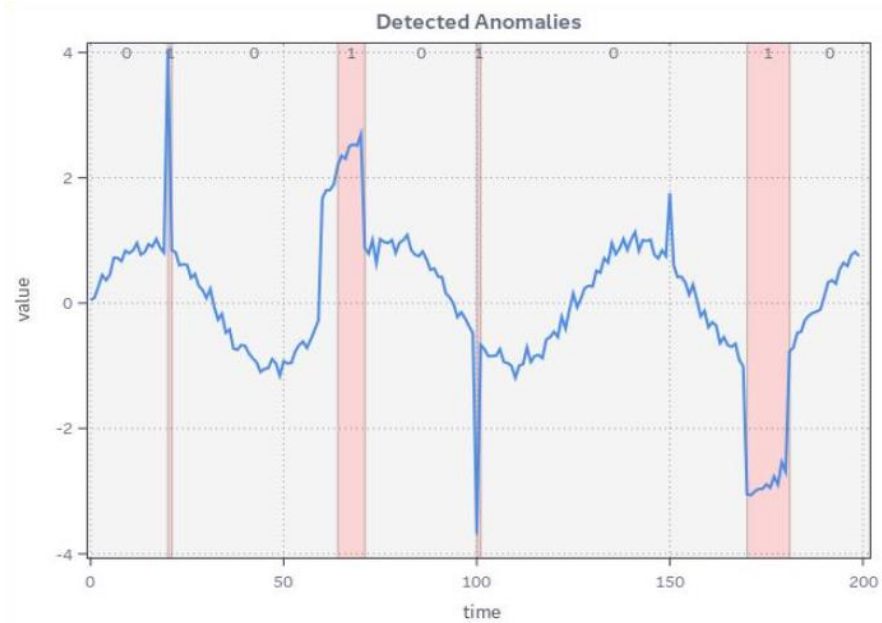
- Anomaly Detection: 이미지 상의 이상을 판단함
- Image Segmentation: 이미지를 의미론적 단위로 분할
- Sensor fusion: 다수의 이미지 데이터를 융합

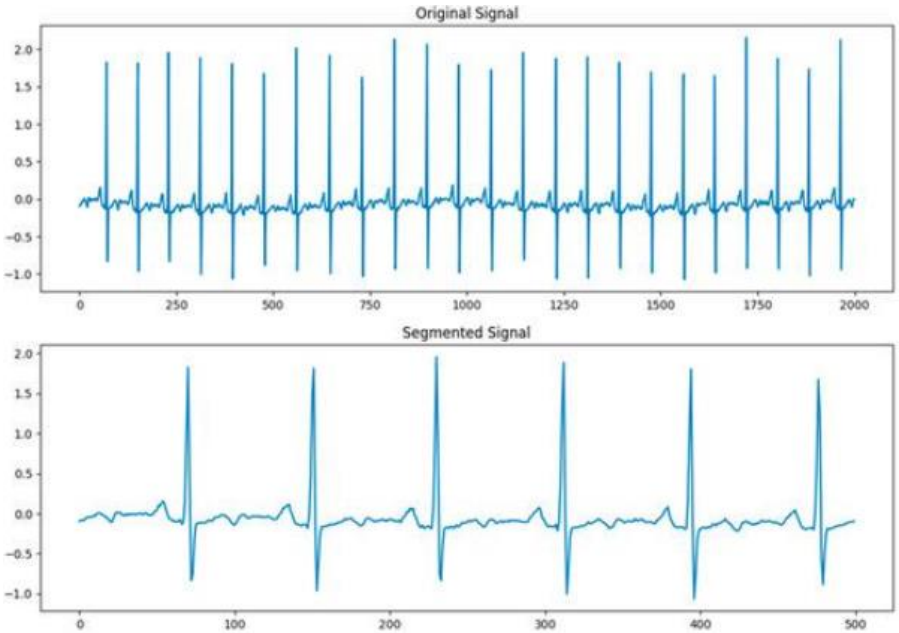
## 1.2. Applications in Deep Learning (Natural Language Processing, NLP)




- 번역 (자동 번역)
- 챗봇 (고객 상담, AI 비서)
- 감정 분석 (긍정/부정 판단)
- 텍스트 요약

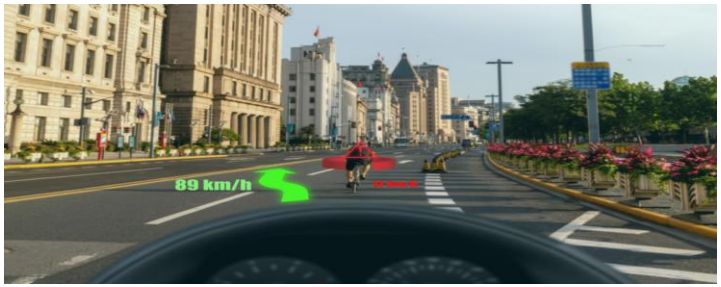
## ➢ 1.2. Applications in Deep Learning (Signal Processing)

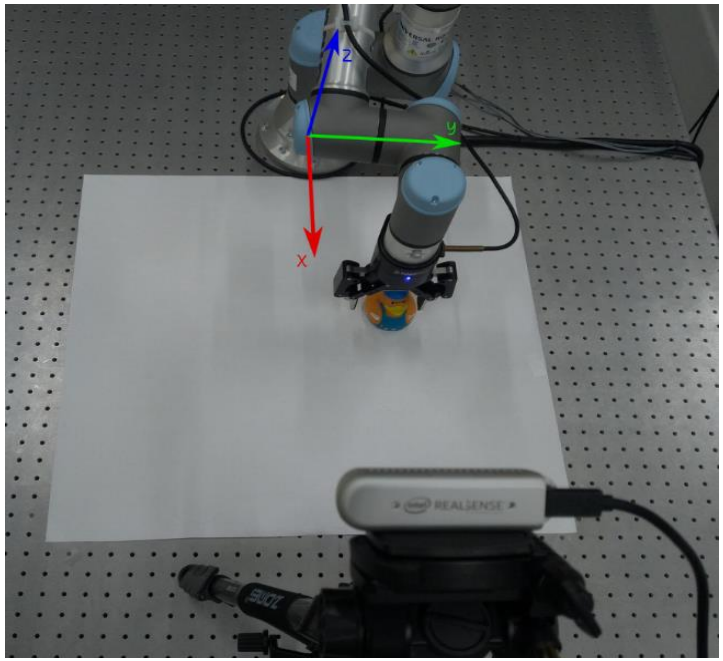



- 상태 예측: 입력 특징으로부터 향후 시점 혹은 수치를 예측
- 이상 탐지: 입력 신호를 기반으로 이상 신호를 구분
- 신호 분석: 데이터의 이상 class를 구분

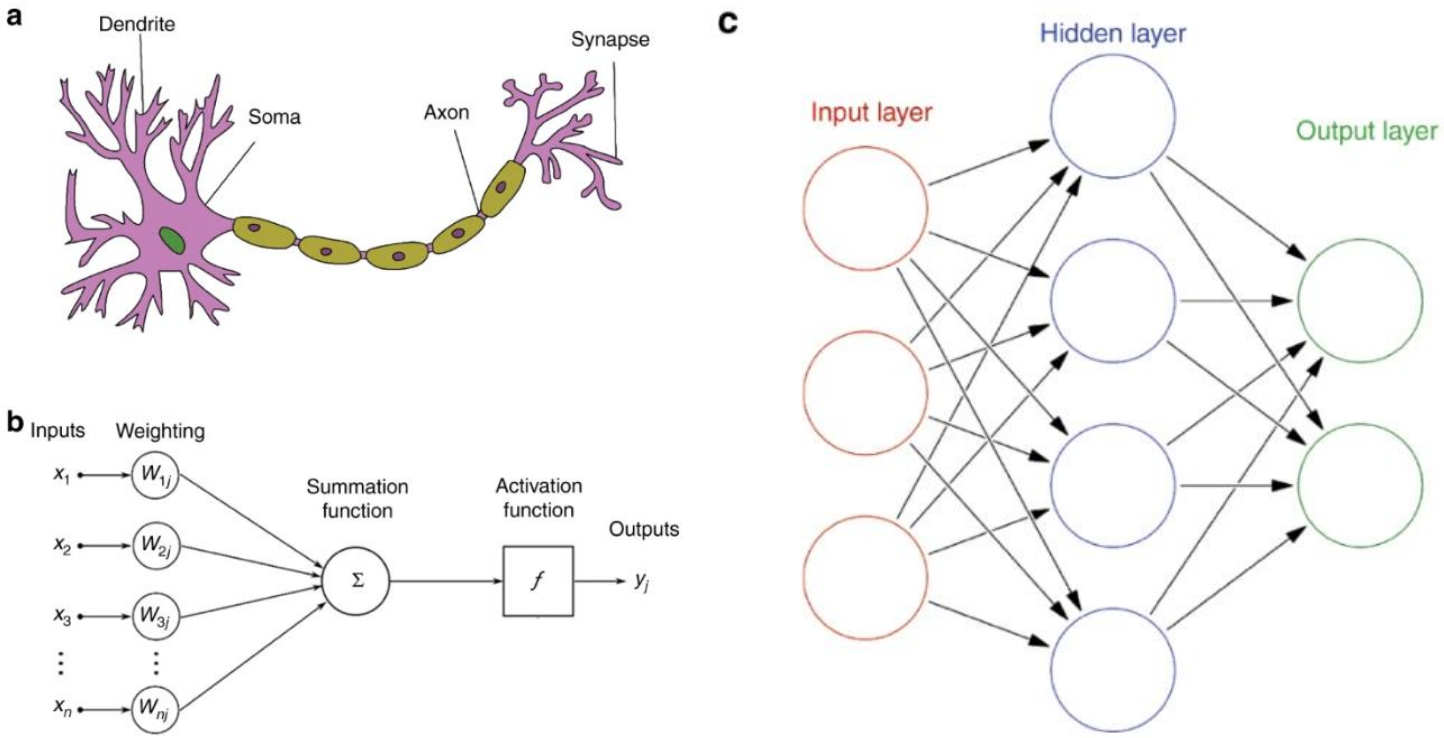
## 1.2. Applications in Deep Learning (Robot Contorl)



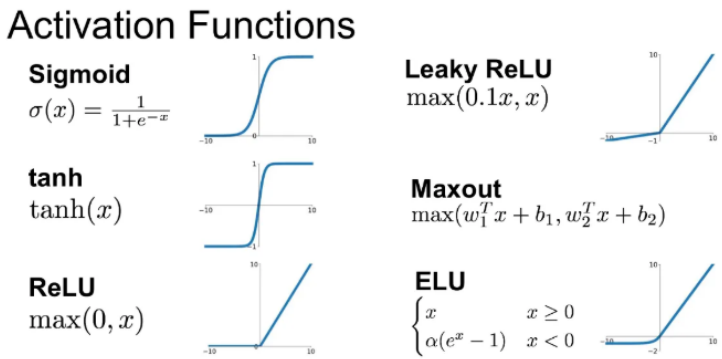

- 환경 인지: 센서를 기반으로 현재 상황에 대한 인지
- 판단: 인지 데이터를 기반으로 한 판단
- 제어: 로봇 구동을 위한 motor 제어

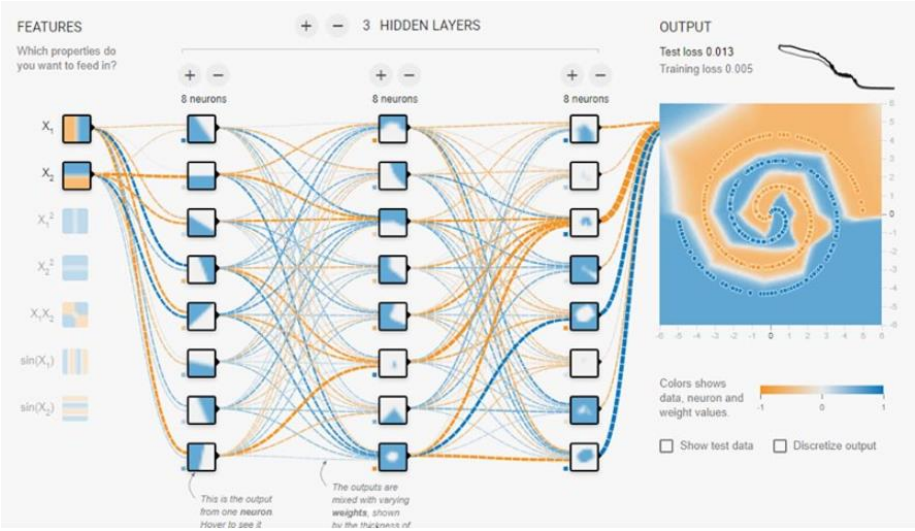
## 1.3 Principle of Deep Learning (Neural network)




- 인공 신경망은 인간의 신경 세포(a)를 착안하여 제안되었으며, 뉴런(b)를 기본 단위로 함
- 뉴런은 이전 계층에서 연결된 뉴런에 가중치를 곱한 후 취합하며, Activation Function을 활용해 다음 층의 뉴런으로 전달
- 즉, 신경이 정보를 전달하는 과정과 유사하게 다른 신경으로부터 정보를 받아드리고 전달하는 과정을 인공적으로 수행할 수 있음

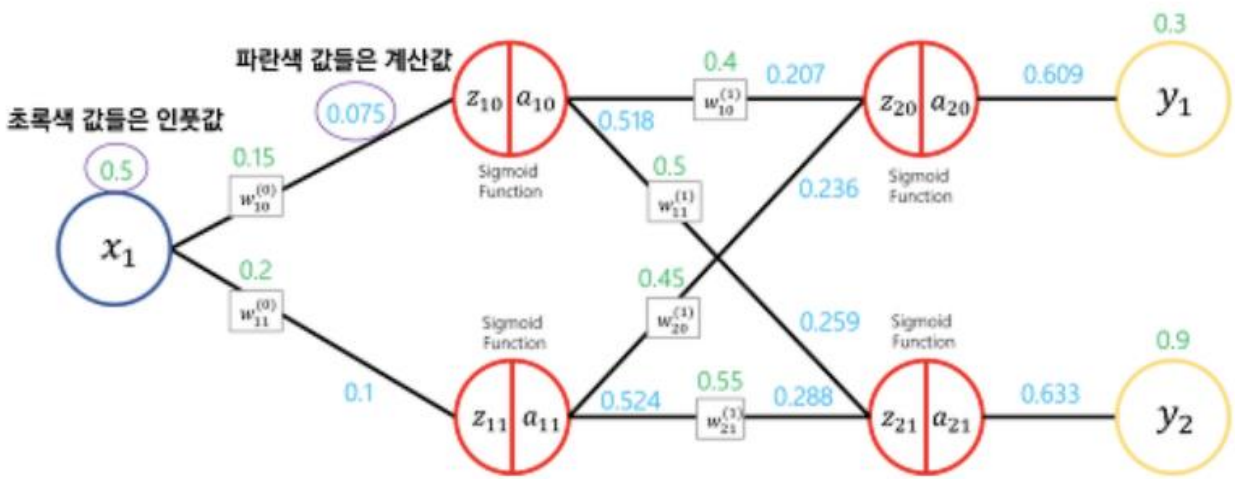
## ➢ 1.3 Principle of Deep Learning (Activation Function)




- Activation Function을 통해 네트워크가 비선형적으로 더 복잡하며, 유연한 특징을 학습할 수 있도록 함
- Activation Function을 활용하지 않는다면 선형적인 특징만을 학습하게 됨
- 비선형 함수가 사용되지 않는다면 선형 연산만 이루어지므로 네트워크를 깊게 쌓는 것이 비효율적임

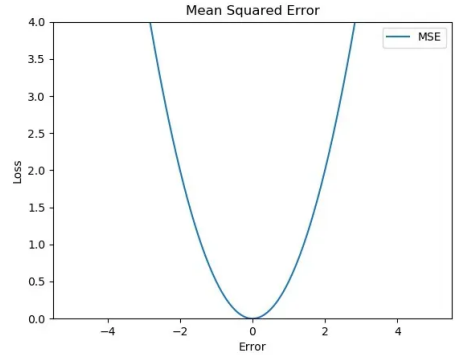
## ➢ 1.3 Principle of Deep Learning (Inference-Feedforward)

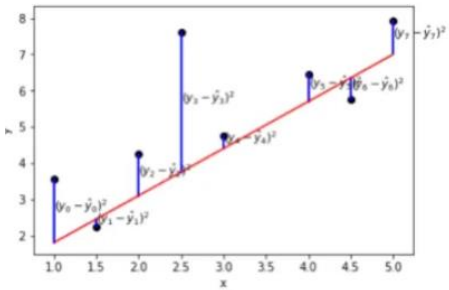



- 딥 러닝 모델의 추론은 입력이 신경망을 따라 전방으로 흐르며 출력이 계산되는 과정을 통해 결과 값을 출력함
- 모델의 출력은 신경망을 구성하고 있는 Parameter(가중치, 편향), Activation Function의 연산을 통해 이루어짐
- 목적에 맞게 동작하기 위해서는 네트워크를 구성하고 있는 Parameter의 값이 중요하며, 최적의 Parameter 값이 설정될 경우 안정적인 추론이 가능

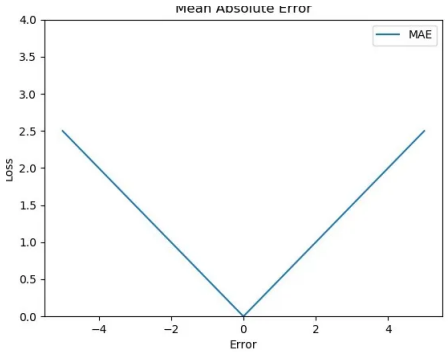
## ➢ 1.3 Principle of Deep Learning (Training-Loss Function)

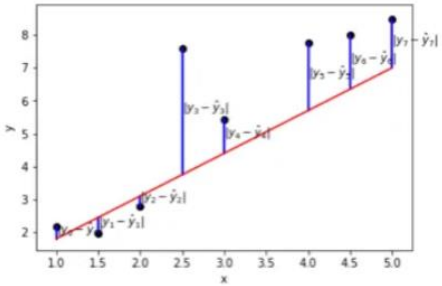
$$MSE = \frac{1}{N}\sum_{i}^{N}(pred_i - target_i)^2$$



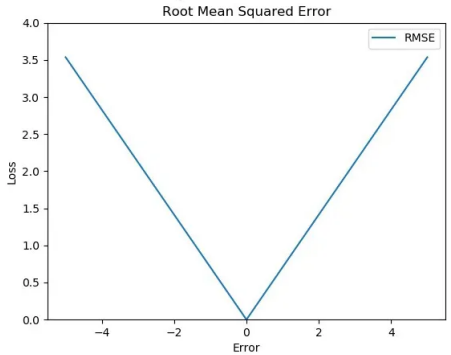



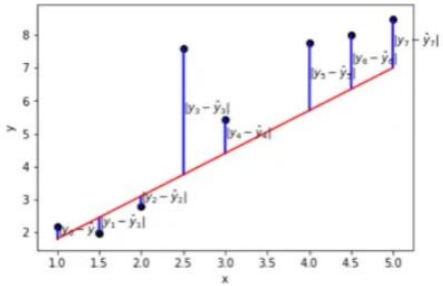
$$MAE = \frac{1}{N}\sum_{i}^{N}|(pred_i - target_i)|$$



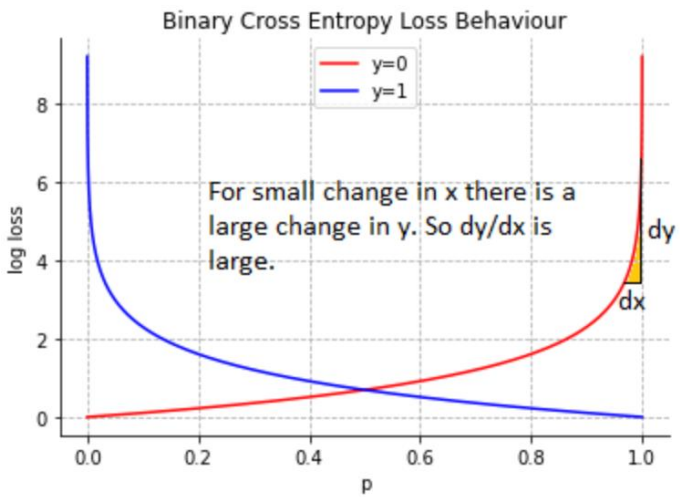
$$RMSE = \sqrt{\frac{1}{N}\sum_{i}^{N}(pred_i - target_i)^2}$$

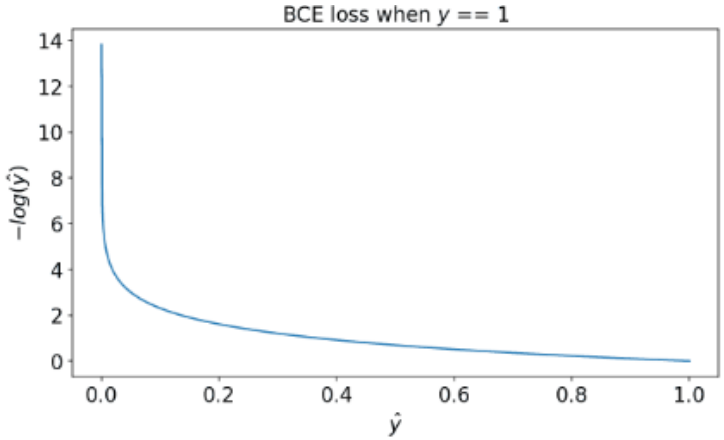


- 모델 학습 과정의 평가 지표로서, 인공 신경망이 예측한 값이 실제 값에 대해 얼마나 잘 예측했는지를 판단함.
- 예측: MSE, MAE, RMSE … / 분류: BCE, CE

## ➢ 1.3 Principle of Deep Learning (Training-Loss Function)



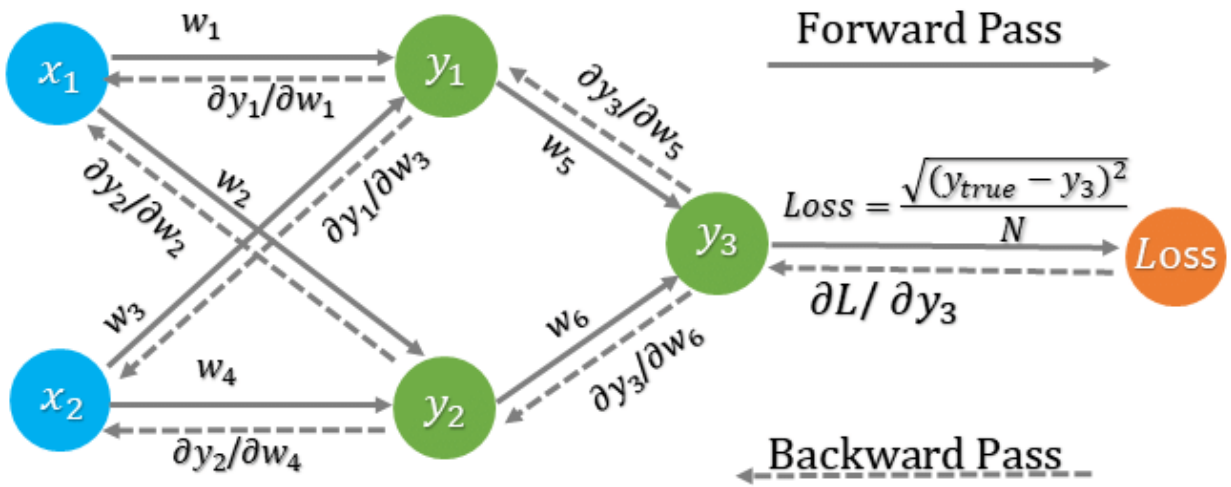




$$BCE = \begin{cases} -log(\hat{y}), where\ y = 1 \\ -log(1-\hat{y}), where\ y = 0 \end{cases}$$
$$= -\left[ylog(\hat{y}) + (1-y)log(1-\hat{y})\right]$$

$$H(P,Q) = -\mathbb{E}_{X\sim P}[\log Q(x)] = -\sum P(x)\log Q(x)$$

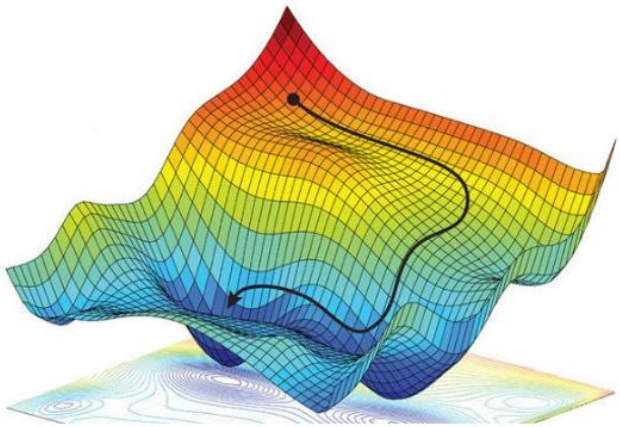
- 모델 학습 과정의 평가 지표로서, 인공 신경망이 예측한 값이 실제 값에 대해 얼마나 잘 예측했는지를 판단함.
- 예측: MSE, MAE, RMSE … / 분류: BCE, CE

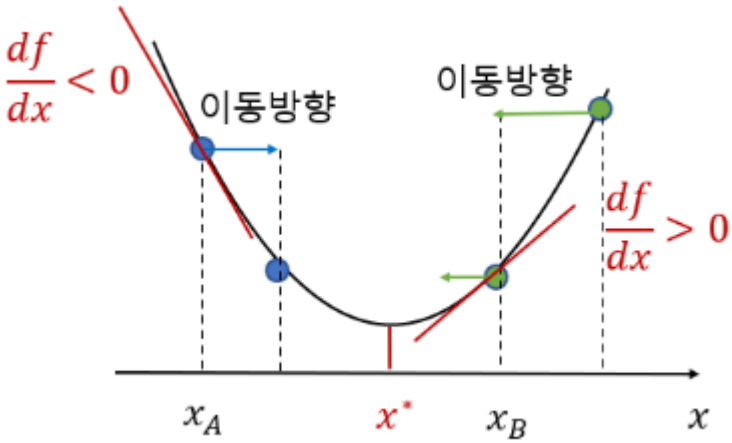
## ➢ 1.3 Principle of Deep Learning (Training-Back Propagation & Chain Rule)




- 인공 신경망을 구성하고 있는 Parameter의 최적 값을 찾는 과정은 Back Propagation, Chain Rule을 통해 이루어짐.
- 순전파를 통해 추론된 인공신경망의 결과와 Label을 Loss Function을 통해 Error 값을 산출하며, Error 값이 감소하는 방향으로 Parameter 값을 업데이트 함.

## ➢ 1.3 Principle of Deep Learning (Training-Back Propagation & Chain Rule)

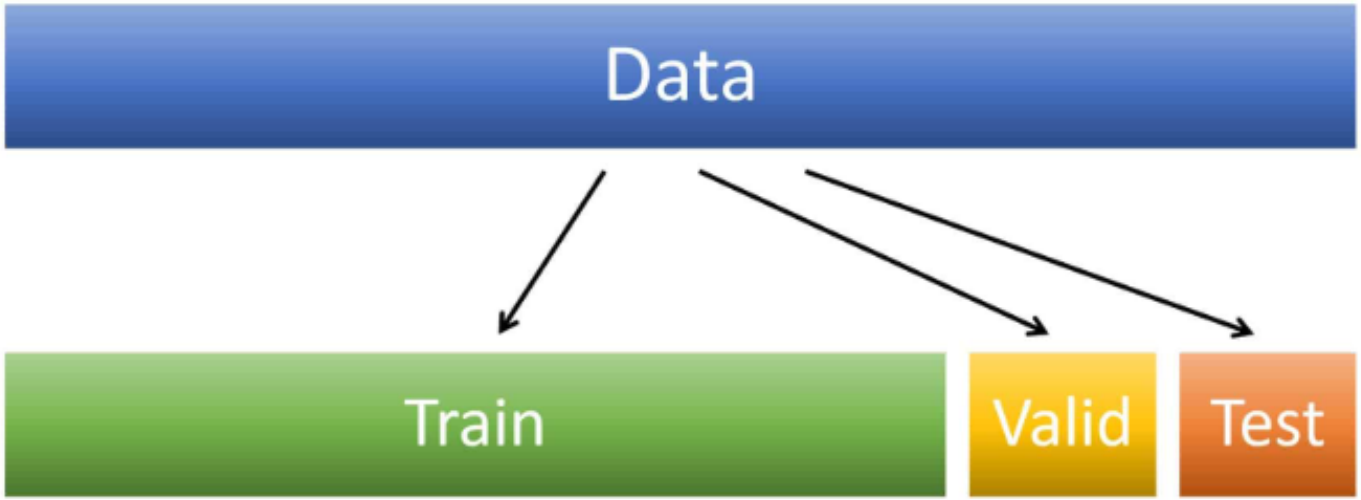


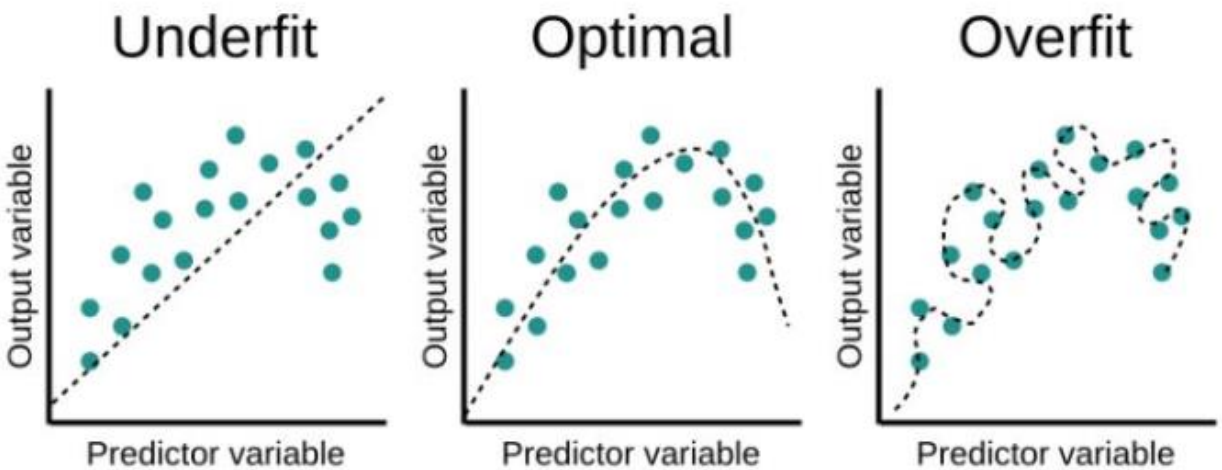



$$\theta := \theta - \alpha \nabla_\theta J(\theta)$$

- 인공 신경망을 구성하고 있는 Parameter의 Update는 Gradient decent를 활용하여 최적화됨.
- 비용함수(Cost Function)을 신경망을 이루고 있는 Parameter로 편미분하여, 미분 값의 반대방향으로 값을 업데이트

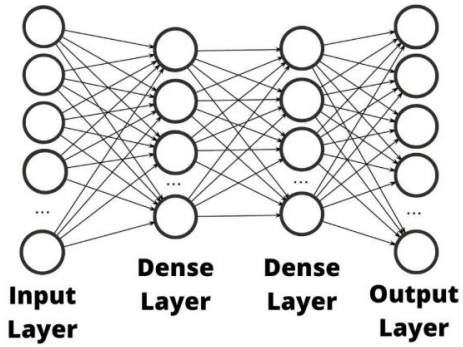
## 1.3 Principle of Deep Learning (Training-Overfitting, Underfitting)

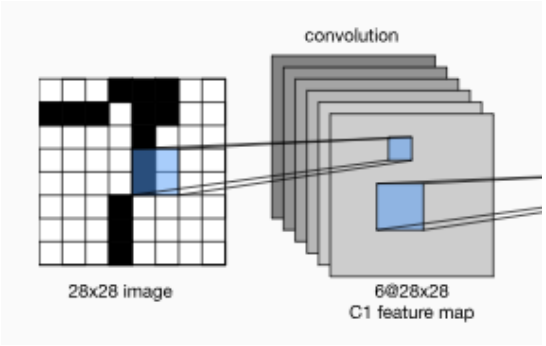


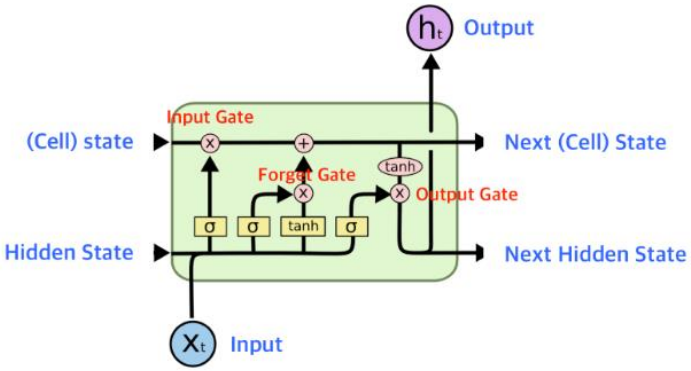



# 2. Basic Network Structure

## ➢ 2.1. Basic Network



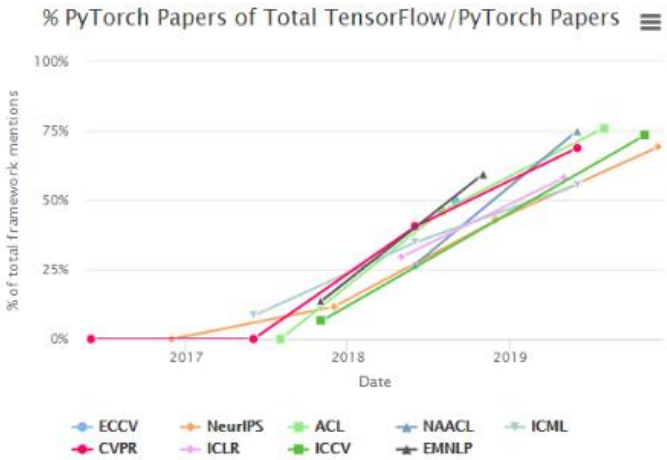



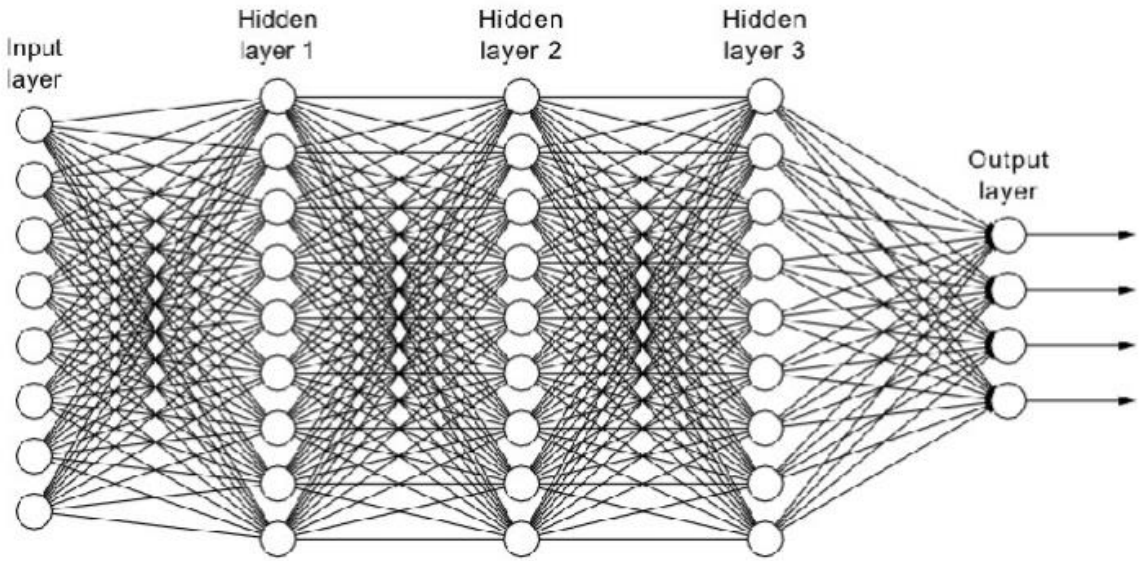

## ➢ 2.2. Programing

TensorFlow

PYTORCH

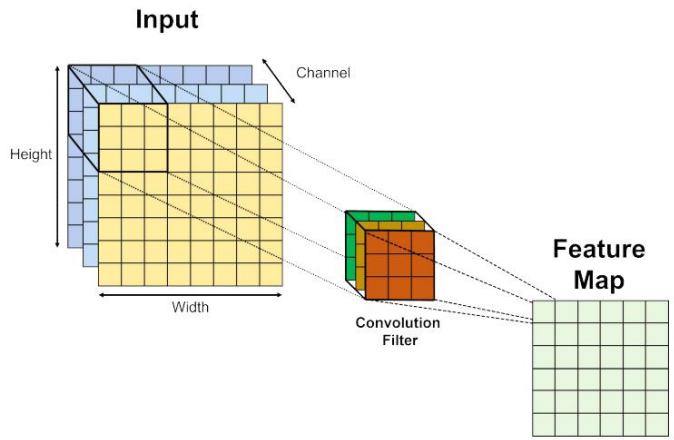


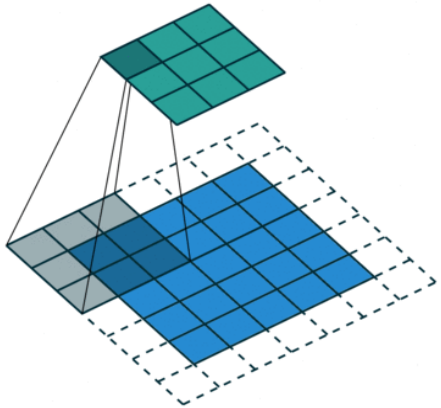
## ➢ 2.3. Dense (Fully Connected Network)

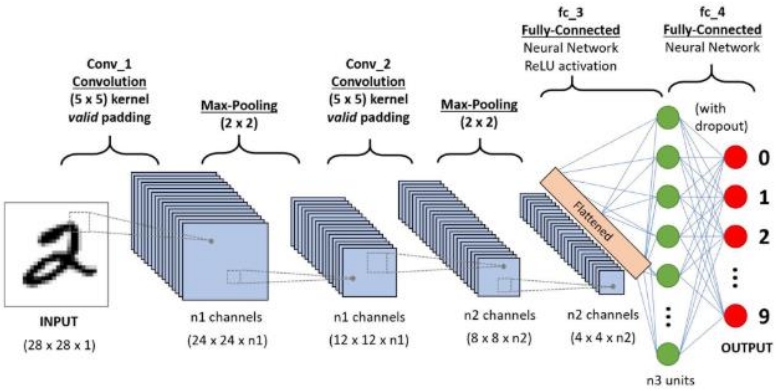



- 이전 층을 구성하고 있는 노드는 다음 층을 구성하는 노드와 모두 연결 됨
- 파라미터 수가 많으며, 과대 적합의 위험성 등이 존재하나, 특정 어플리케이션에서는 여전히 사용되고 있는 신경망임
- Dense Layer 만을 사용하여 구성된 네트워크는 많이 사용되지 않으나, 다양한 네트워크에 접목되어 사용되고 있음.

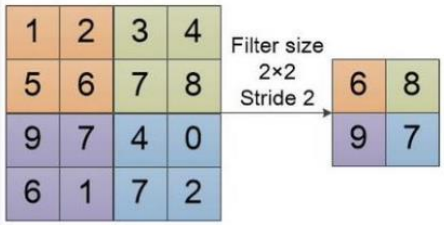
## ➢ 2.4. CNN



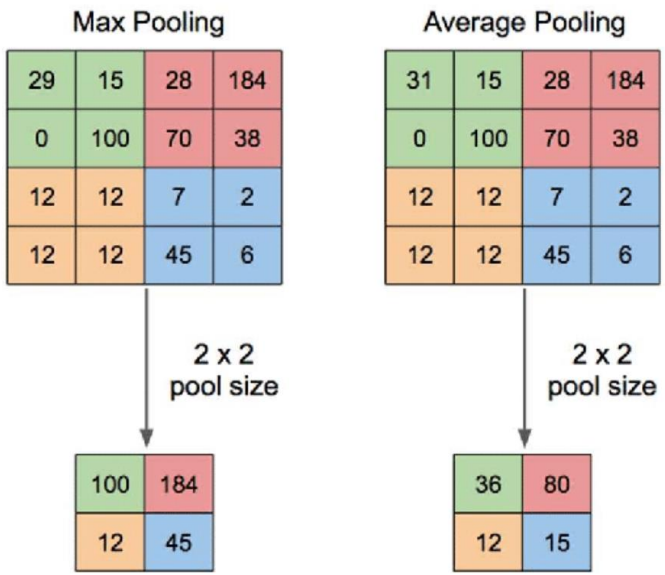



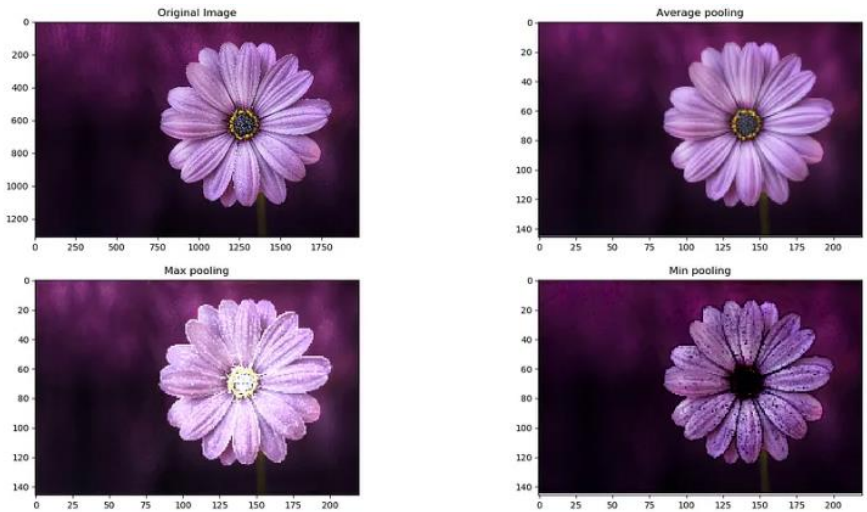



$$O = \frac{I - F + 2P}{S} + 1$$

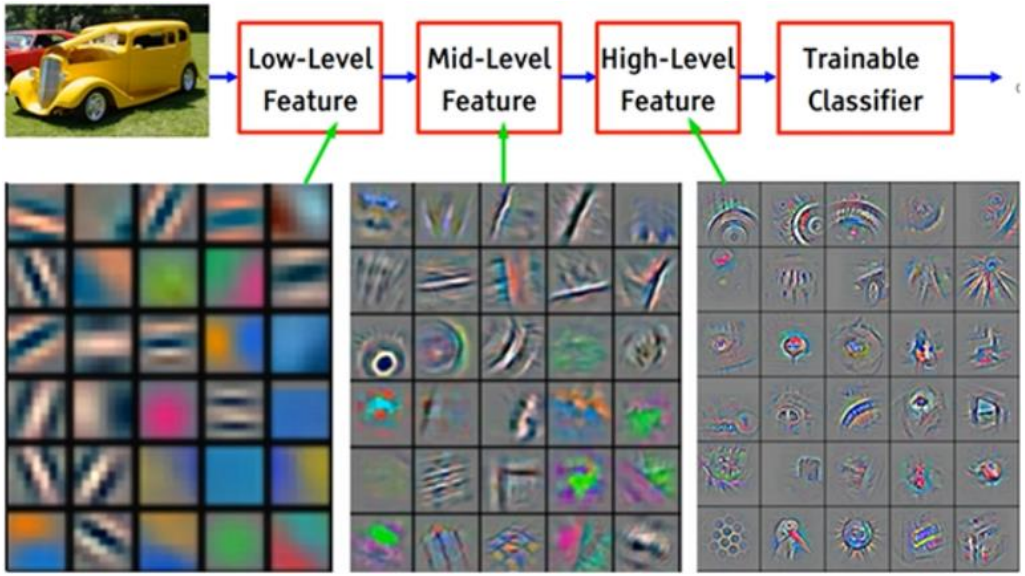


## ➢ 2.4. CNN


Max Pooling
29 15 28 184
0 100 70 38
12 12 7 2
12 12 45 6
2 x 2
pool size
100 184
12 45
Average Pooling
31 15 28 184
0 100 70 38
12 12 7 2
12 12 45 6
2 x 2
pool size
36 80
12 15

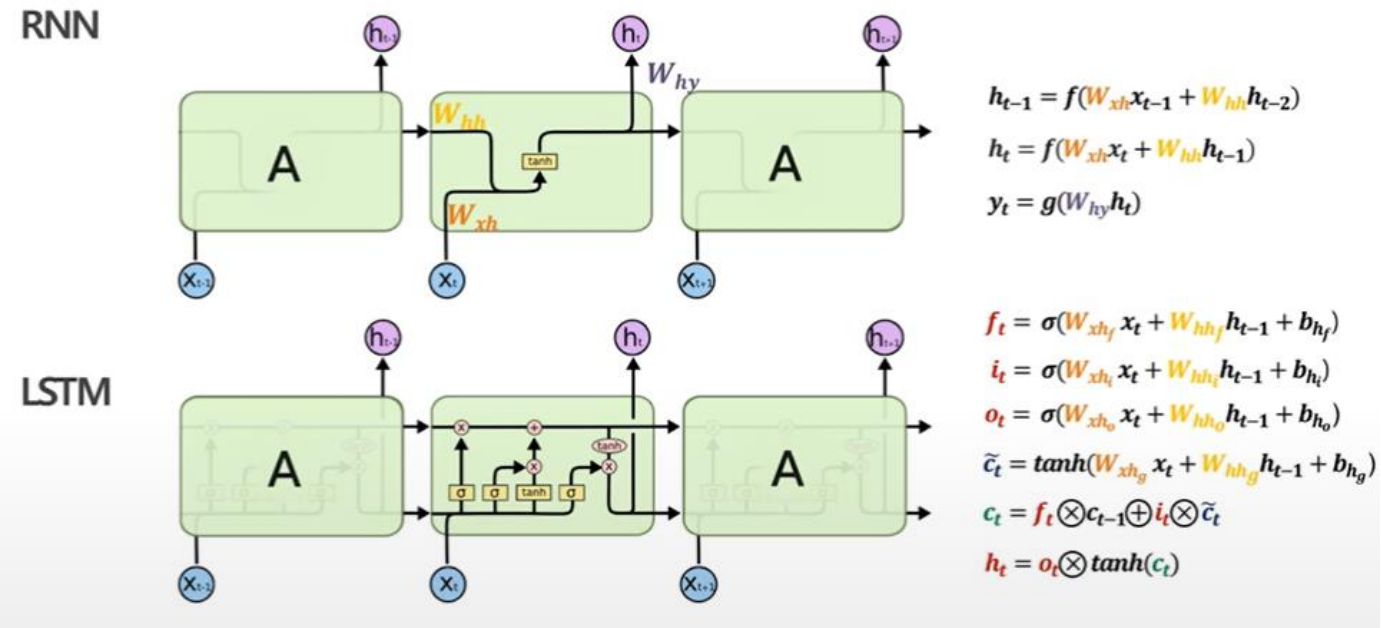

Original Image
Average pooling
Max pooling
Min pooling

## ➢ 2.4. CNN



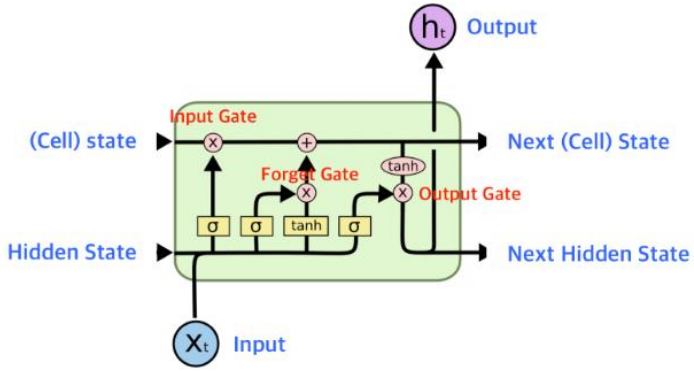

- Filter를 통해 입력을 슬라이딩 하며 공간적인 상관관계를 추출함.
- 파라미터 수가 작다는 장점을 가지며, 다양한 네트워크에서 사용되고 있음.
- Channel의 특징을 조합하여 새로운 특징을 생성함.

## ➢ 2.5. RNN & LSTM

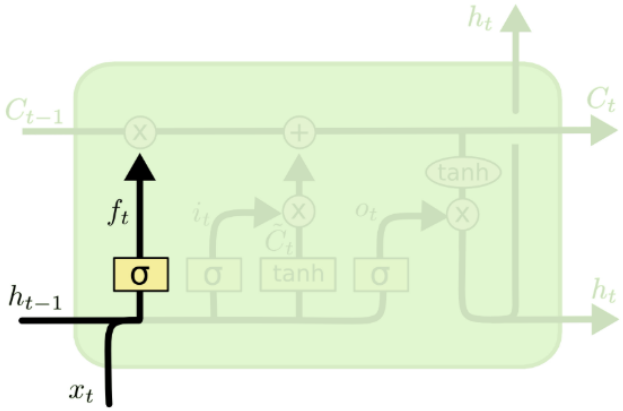



- RNN은 시계열 데이터에 적용되어 안정적인 성능을 보여주었으나, 파라미터가 업데이트 되는 과정에서 1보다 작은 값이 계속 곱해짐으로 기울기 소멸 문제가 발생
- 이에 따라, 망각 게이트(forget gate), 입력 게이트(input gate), 출력 게이트(output gate)라는 요소를 추가함.

## ➢ 2.5. RNN & LSTM




과거의 정보를 어느 정도 기억할 것인가?

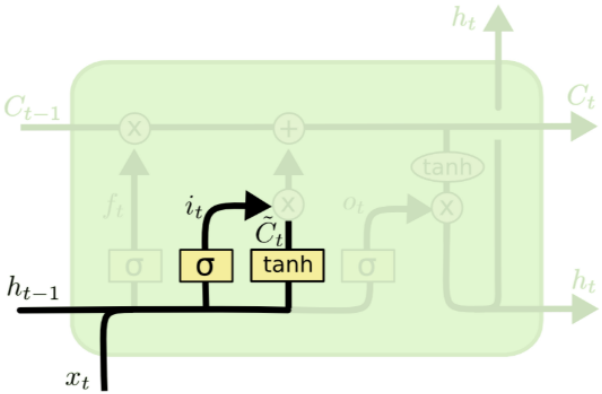



[Figure 3] 망각 게이트

$$f_t = \sigma(W_f h_{t-1} + W_f x_t + b_f)$$

- input: 과거의 정보 $h_{t-1}$, 새로운 입력값 $x_t$
- output: 과거의 정보를 반영하는 비중 $f_t \in (0, 1)$
- $\sigma$: sigmoid function

현재 정보(새로운 정보)를 어느 정도 기억할 것인가?




[Figure 4] 입력 게이트

[Input gate layer]

$$i_t = \sigma(W_i h_{t-1} + W_i x_t + b_i)$$

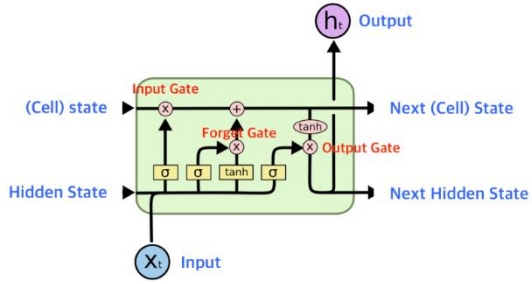
- input: 과거의 정보 $h_{t-1}$, 새로운 입력값 $x_t$
- output: 현재의 정보를 반영하는 비중 $i_t \in (0, 1)$

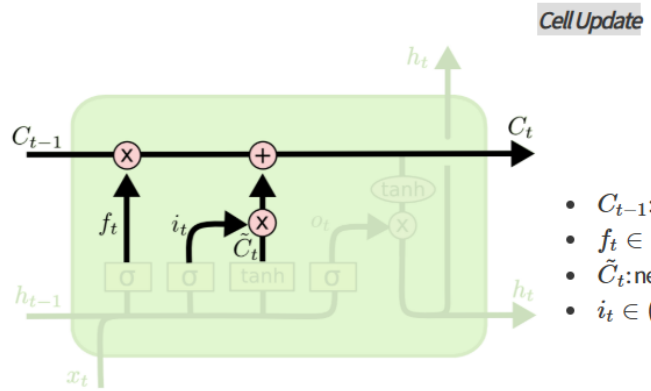
[Tanh layer]

$$\tilde{C}_t = \tanh(W_c h_{t-1} + W_c x_t + b_c)$$

- input: 과거의 정보 $h_{t-1}$, 새로운 입력값 $x_t$
- output: new candidate values $\tilde{C}_t$

## ➢ 2.5. RNN & LSTM

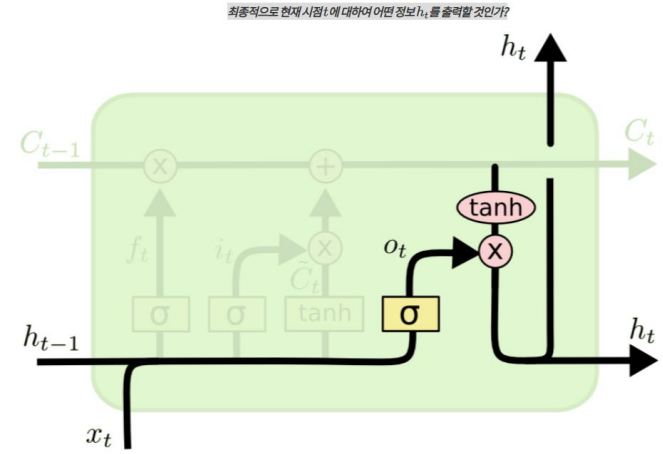



[Figure 5]: Cell

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

- $C_{t-1}$: old cell state
- $f_t \in (0, 1)$: 과거 정보 반영 비중
- $\tilde{C}_t$: new information(새로 업데이트된 정보)
- $i_t \in (0, 1)$: 현재 정보(새로운 정보) 반영 비중




[Figure 6]: Output gate

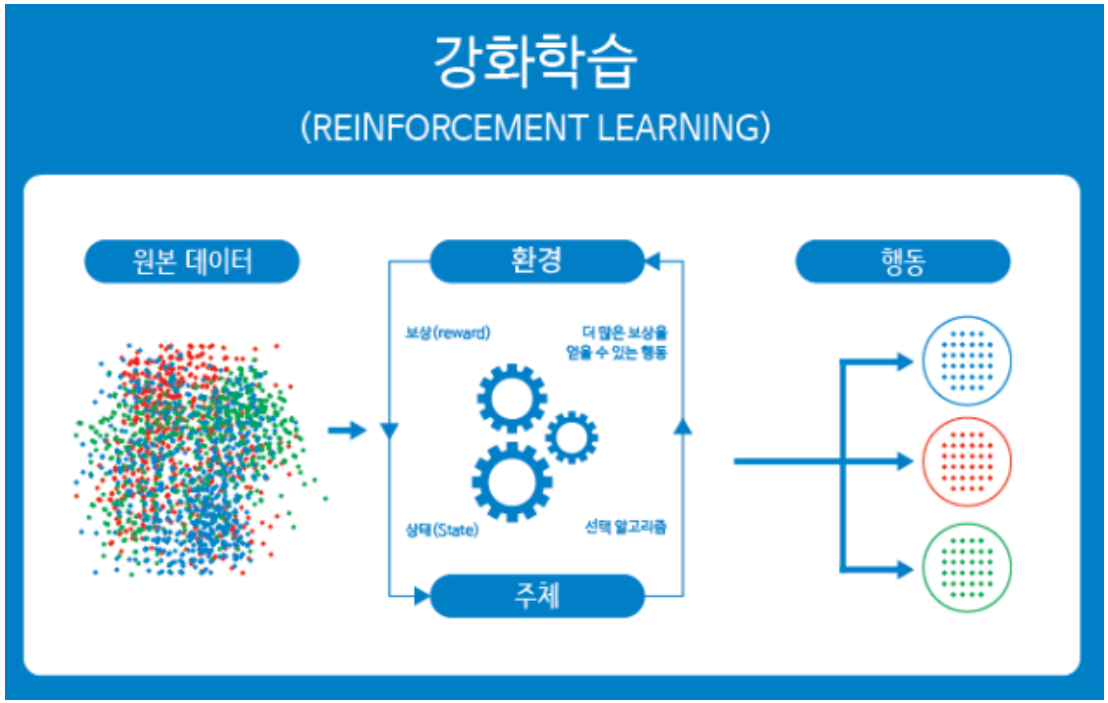
$$o_t = \sigma(W_o h_{t-1} + W_o x_t + b_o)$$

- input: 과거의 정보 $h_{t-1}$, 새로운 입력값 $x_t$
- output: cell state의 어떤 정보(부분)을 반영할지에 대한 비중 $o_t \in (0, 1)$

$$h_t = o_t \tanh(C_t)$$

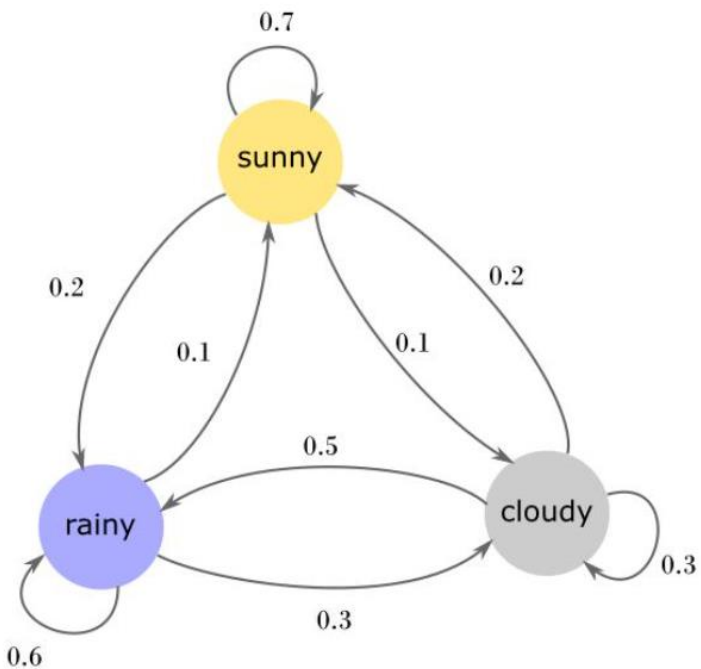
- $h_t$: 입력값 $h_{t-1}$과 $x_t$에 대한 최종 출력값(output)

## ➢ 3.1. Define of Reinforcement




- 강화학습은 에이전트(Agent)가 환경(Environment)과 상호작용하면서, 각 상황에서 어떤 행동(Action)을 선택해야 장기적인 보상(Reward)을 최대화할 수 있는지를 학습하는 방법
- 즉, 강화학습은 누적 보상을 최대화하기 위해 시행착오를 통해 행동을 교정해 나가는 학습 과정으로 볼 수 있음
- 강화 학습은 순차적 의사결정 문제를 푸는 방법론임

## 3.2. Markov Process




**Markov Process :** $\boldsymbol{MP} \equiv (\boldsymbol{S}, \boldsymbol{P})$

**상태 집합 S**

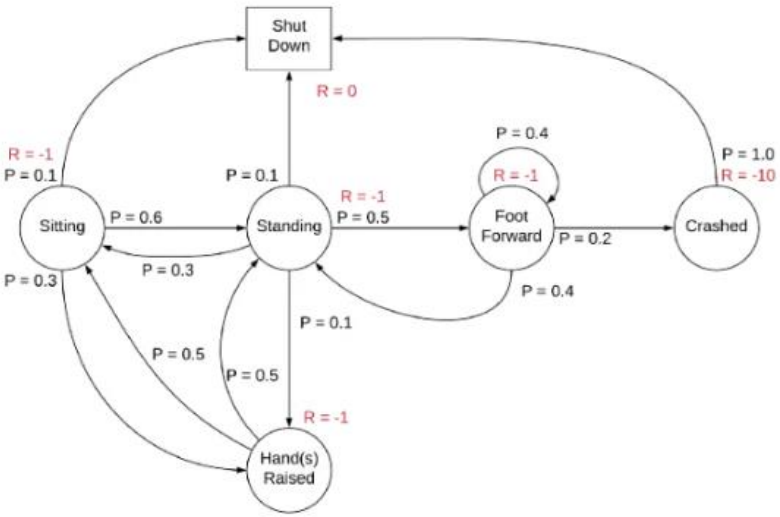
- 가능한 상태의 집합 S = {sunny, rainy, cloudy}

**전이 확률 행렬 P**

- $P_{cloudy\ rainy}$: cloudy 상태에서 rainy가 될 확률
- $P_{cloudy\ rainy} = P[s_{t+1} = rainy \mid s_t = coludy]$

- Markov Process는 미리 정의된 확률에 따라 상태 전이가 일어나는 과정을 의미함
- 특정한 상태에 도달하면 다음 상태가 어떻게 될지에 대한 확률이 존재하며, 각 확률에 따라 다음 상태가 정해짐
- 즉, 현재 상태만 알면 미래를 예측할 수 있는 확률 과정이며, 미래는 과거가 아니라 현재 상태에만 의존함

## ➢ 3.2. Markov Reward Process




- MRP에서 종료 상태에 도달하기까지 어떤 상태를 거쳐야 할지는 확률 적으로 정해지며, 이는 예측하기 매우 어려움
- 시작부터 종료 시점까지의 과정을 에피소드라고 함

**Markov Process :** $\boldsymbol{MRP} \equiv (\boldsymbol{S}, \boldsymbol{P}, \boldsymbol{R}, \boldsymbol{\gamma})$

**상태 집합 S**
- 가능한 상태의 집합

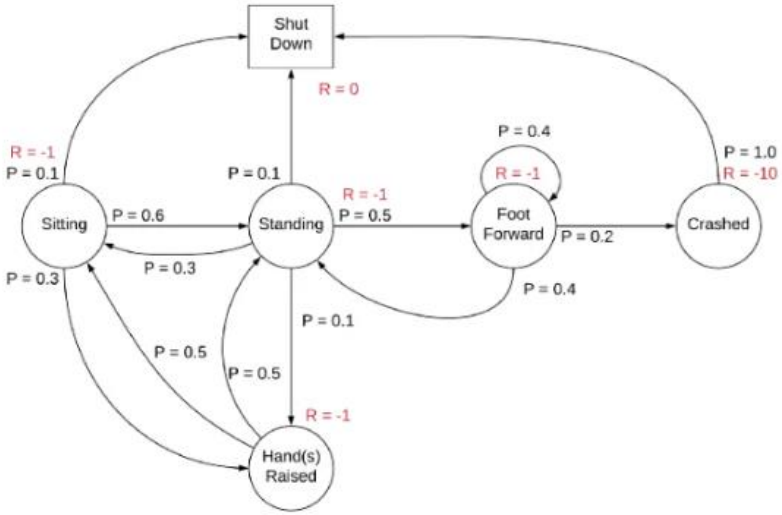
**전이 확률 행렬 P**
- 마르코프 프로세스의 P와 같으며, 상태 s에서 상태 s'로 갈 확률을 행렬 형태로 표시

**보상 함수 R**
- 어떤 상태 S에 도착했을 때 받게 되는 보상을 의미
- $R = E[R_t | S_t = s]$

**리턴 $G_t$**
- 상태 $s_0$에서 $R_0$의 보상을 받고 종료 상태인 $s_t$에서 $R_t$의 보상을 받고 종료됨
- $G_t$는 t시점부터 미래에 받을 감쇠된 보상 합을 의미함

**감쇠 인자 $\gamma$**
- 강화학습에서 미래에 얻을 보상에 비해 당장 얻는 보상을 얼마나 더 중요하게 여길 것인지를 나타내는 파라미터
- 미래에 받을 보상에 $\gamma$가 여러번 곱해지면서 그 값을 작게 만드는 역할

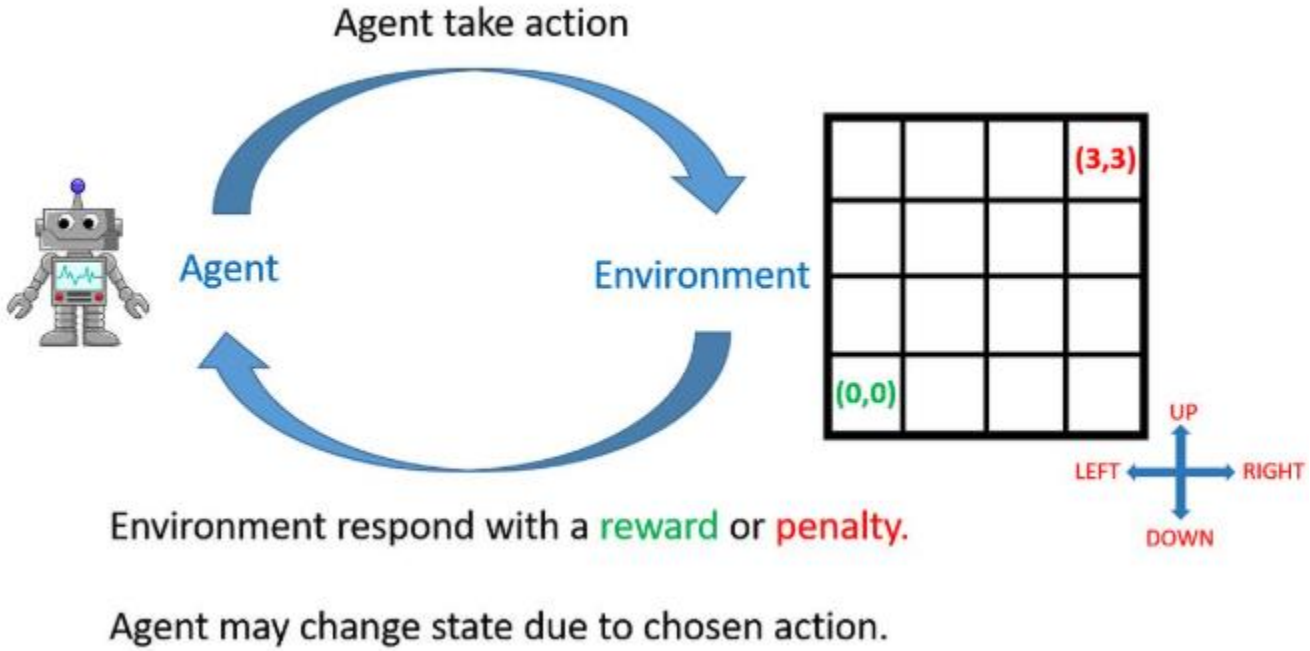
## 3.2. Markov Reward Process




**상태 가치 함수(State Value Function)**

- 상태 가치 함수는 상태를 인풋으로 넣으면 그 상태의 밸류를 아웃풋으로 출력하는 함수
- 에피소드마다 리턴이 다르기 때문에 어떤 상태 s에 대한 상태 가치는 기대값을 이용해 정의함
- Monte-carlo 접근법에 따라, 에피소드를 샘플링하여 리턴을 산출하고 이에 대한 평균을 통해 기대 값을 산출함

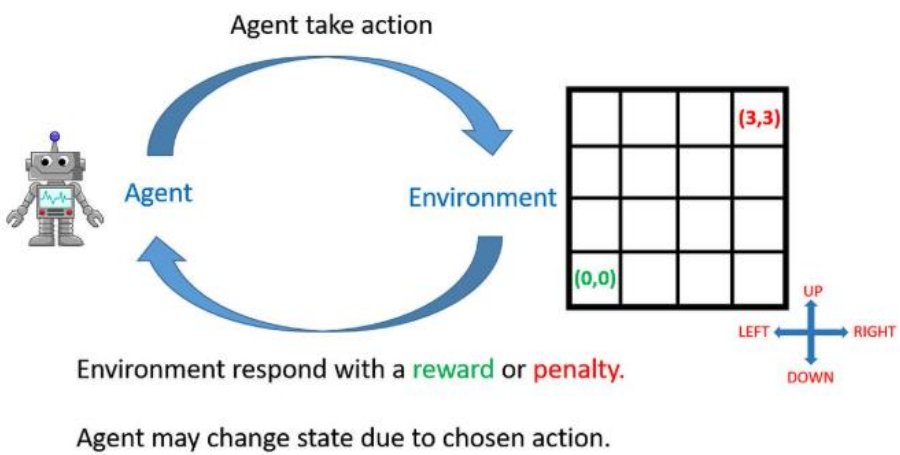
## 3.3. Markov Decision Process




**Markov Decision Process :** $\boldsymbol{MDP} \equiv (\boldsymbol{S}, \boldsymbol{A}, \boldsymbol{P}, \boldsymbol{R}, \boldsymbol{\gamma})$

- 자신의 의사를 가지고 행동하는 주체인 에이전트를 통해 상태 전이가 발생함
- MDP에서는 에이전트가 MRP에 더해진 형태이며, 에이전트는 각 상황에 따라 액션을 하며, 이로 인해 상태가 변화하고 보상을 받게 됨

## 3.3. Markov Decision Process




**상태 집합 S**

- 가능한 상태의 집합

**액션 집합 A**

- 에이전트가 취할 수 있는 액션들을 모아 놓은 집합
- 에이전트는 액션 집합에서 하나를 선택해 액션을 취하며 이에 따라 다음 상태가 달라짐

**전이 확률 행렬 P**

- MDP에서는 액션에 따라 상태가 달라지며, 같은 상태에서 동일한 행동을 한다 하더라도 다음 상태는 달라질 수 있음
- $P_{ss'}^{a} = P[S_{t+1} = s' | S_t = s,\ A_t = a]$

**감쇠 인자 $\gamma$**

- 강화학습에서 미래에 얻을 보상에 비해 당장 얻는 보상을 얼마나 더 중요하게 여길 것인지를 나타내는 파라미터
- 미래에 받을 보상에 $\gamma$가 여러번 곱해지면서 그 값을 작게 만드는 역할

**보상 함수 R**

- 상태 $s$에서 액션 $a$를 했을 때, 받는 보상
- 확률적으로 보상이 달라지므로 기대값으로 정의함
- $R_{s}^{a} = E[R_{t+1} | S_t = s,\ A_t = a]$

## 3.3. Markov Decision Process

- MDP에서 결국 찾고자 하는 것은 각 상태 s에서 어떤 액션 a를 선택해야 보상의 합을 최대로 할 수 있는가?

**정책 함수**

- 정책함수는 각 상태에서 어떤 액션을 선택할지 정해주는 함수
- $\pi(a|s) = P[A_t = a \mid S_t = s]$

**상태 가치 함수**

- 에이전트가 어떤 행동을 취하는가에 따라 즉, 에이전트의 정책 함수에 따라서 얻는 리턴이 달라짐
- 이에 따라, 정책 함수가 반영된 상태 가치를 평가함
- $v_\pi = E_\pi[r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ \ldots | S_t = s] = E_\pi[G_t | S_t = s]$

**액션 가치 함수**

- 각 상태에서 액션을 평가하는 함수
- 에이트가 어떤 정책을 따르냐에 따라 이후 상태가 달라지므로, 정책을 고정시킨 후 가치를 판단하여야 함
- $q_\pi(s, a) = E_\pi[G_t | S_t = s, A_t = a]$
- 상태 s에서 a를 강제로 선택한 후 가치를 판단 함